

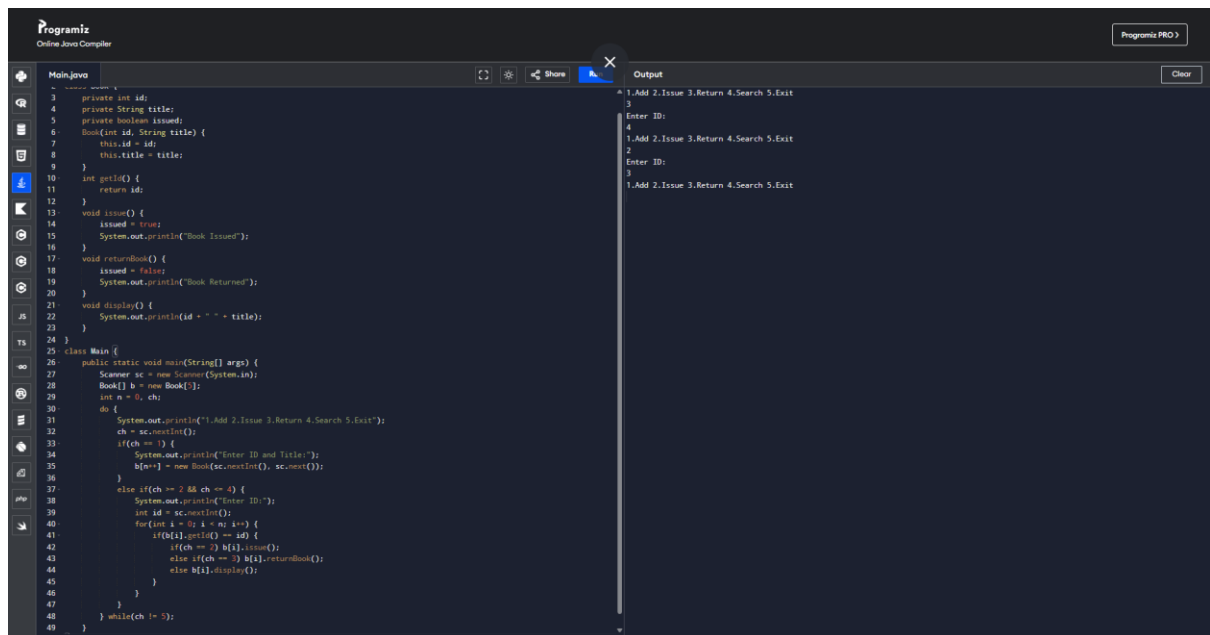
CSA0911

Programming in JAVA

S.Venkata Anirudh(192572110)

-----CO1-AT1-----

1. Library management system

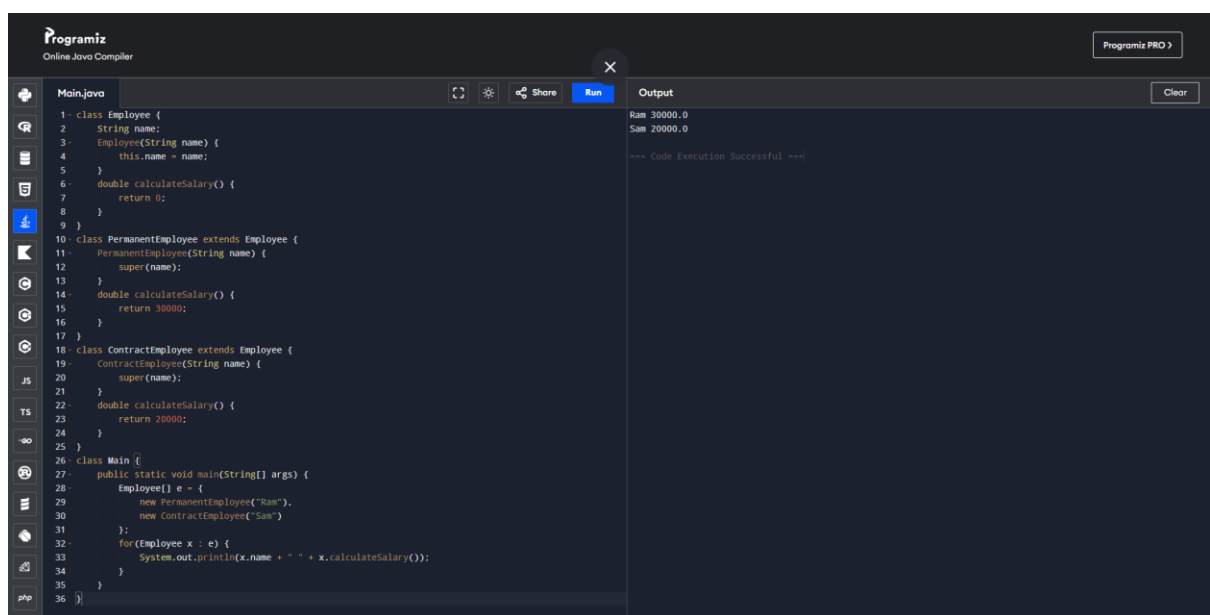


```
1 private int id;
2 private String title;
3 private boolean issued;
4
5 Book(int id, String title) {
6     this.id = id;
7     this.title = title;
8 }
9
10 int getId() {
11     return id;
12 }
13
14 void issue() {
15     issued = true;
16     System.out.println("Book Issued");
17 }
18 void returnBook() {
19     issued = false;
20     System.out.println("Book Returned");
21 }
22 void display() {
23     System.out.println(id + " " + title);
24 }
25
26 class Main {
27     public static void main(String[] args) {
28         Scanner sc = new Scanner(System.in);
29         Book[] b = new Book[3];
30         int n = 0;
31         while (true) {
32             System.out.println("1.Add 2.Issue 3.Return 4.Search 5.Exit");
33             int ch = sc.nextInt();
34             if (ch == 1) {
35                 System.out.println("Enter ID and Title:");
36                 b[n++] = new Book(sc.nextInt(), sc.next());
37             }
38             else if (ch == 2 && ch <= 4) {
39                 System.out.println("Enter ID:");
40                 int id = sc.nextInt();
41                 for (int i = 0; i < b.length; i++) {
42                     if (b[i].getId() == id) {
43                         if (ch == 2) b[i].issue();
44                         else if (ch == 3) b[i].returnBook();
45                         else b[i].display();
46                     }
47                 }
48             }
49             while (ch != 5);
50         }
51     }
52 }
```

Output

```
1.Add 2.Issue 3.Return 4.Search 5.Exit
3
Enter ID:
4
1.Add 2.Issue 3.Return 4.Search 5.Exit
2
Enter ID:
3
1.Add 2.Issue 3.Return 4.Search 5.Exit
```

2. Employee Payroll System

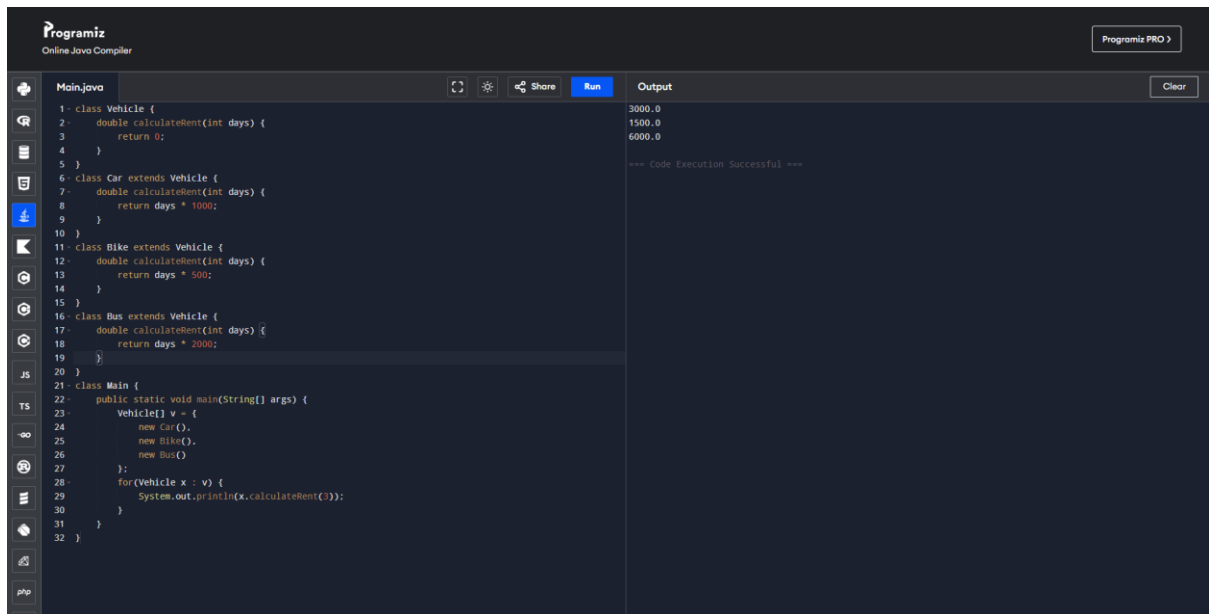


```
1 class Employee {
2     String name;
3     Employee(String name) {
4         this.name = name;
5     }
6     double calculateSalary() {
7         return 0;
8     }
9 }
10 class PermanentEmployee extends Employee {
11     PermanentEmployee(String name) {
12         super(name);
13     }
14     double calculateSalary() {
15         return 30000;
16     }
17 }
18 class ContractEmployee extends Employee {
19     ContractEmployee(String name) {
20         super(name);
21     }
22     double calculateSalary() {
23         return 20000;
24     }
25 }
26 class Main {
27     public static void main(String[] args) {
28         Employee[] e = {
29             new PermanentEmployee("Ram"),
30             new ContractEmployee("Sam")
31         };
32         for (Employee x : e) {
33             System.out.println(x.name + " " + x.calculateSalary());
34         }
35     }
36 }
```

Output

```
Ram 30000.0
Sam 20000.0
--- Code Execution Successful ---
```

3. Vehicle Rental System

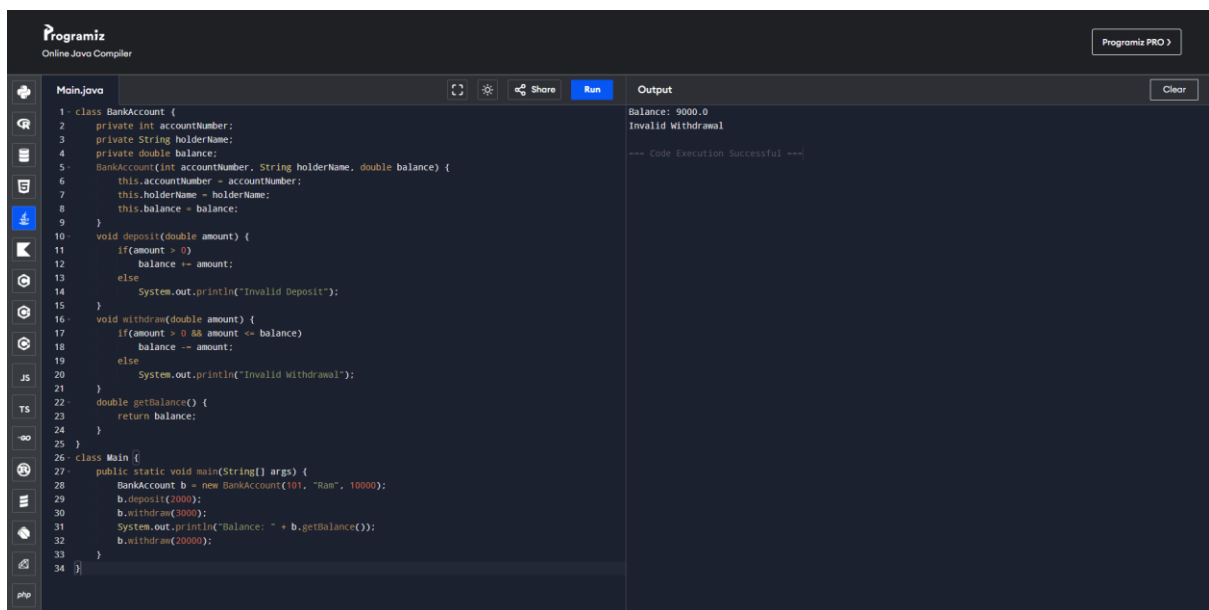


The screenshot shows the Programiz Online Java Compiler interface. The code is written in Java and defines a base class `Vehicle` with a `calculateRent` method. Three subclasses, `Car`, `Bike`, and `Bus`, extend `Vehicle` and override the `calculateRent` method with different rates. The `Main` class creates instances of these vehicles and prints the calculated rent for each.

```
1 class Vehicle {
2     double calculateRent(int days) {
3         return 0;
4     }
5 }
6 class Car extends Vehicle {
7     double calculateRent(int days) {
8         return days * 1000;
9     }
10 }
11 class Bike extends Vehicle {
12     double calculateRent(int days) {
13         return days * 500;
14     }
15 }
16 class Bus extends Vehicle {
17     double calculateRent(int days) {
18         return days * 2000;
19     }
20 }
21 class Main {
22     public static void main(String[] args) {
23         Vehicle[] v = {
24             new Car(),
25             new Bike(),
26             new Bus()
27         };
28         for(Vehicle x : v) {
29             System.out.println(x.calculateRent(3));
30         }
31     }
32 }
```

The output shows the calculated rent for each vehicle: 3000.0 for Car, 1500.0 for Bike, and 6000.0 for Bus. The code execution is successful.

4. Bank Account System

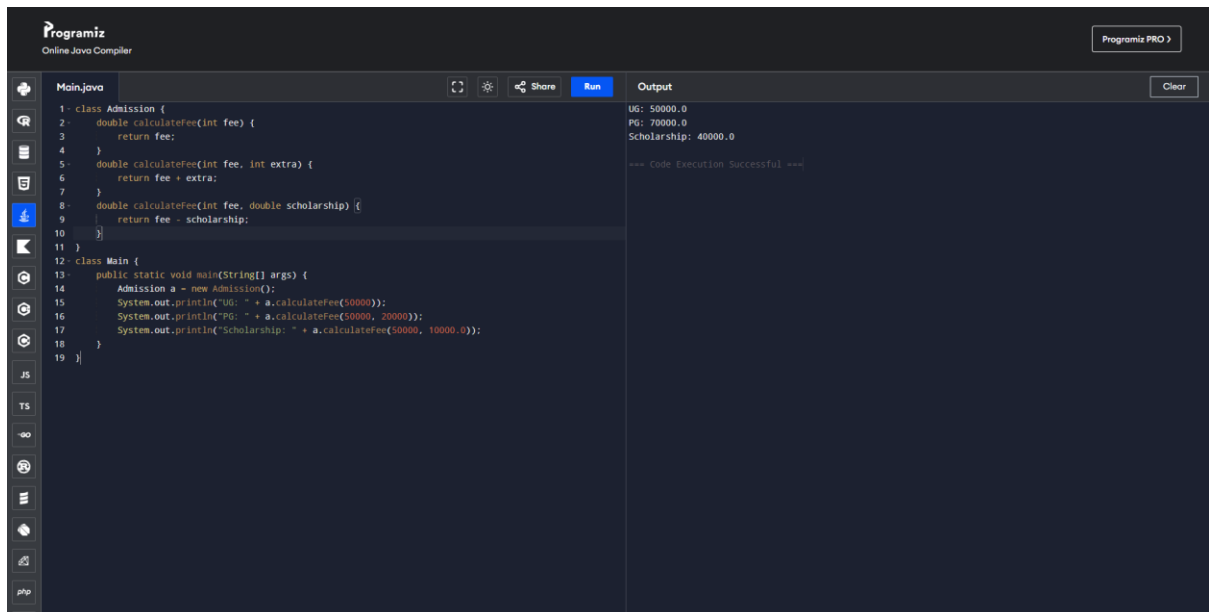


The screenshot shows the Programiz Online Java Compiler interface. The code defines a `BankAccount` class with private attributes `accountNumber`, `holderName`, and `balance`. It includes methods for depositing and withdrawing money, and a method to get the current balance. The `Main` class creates a `BankAccount` object, performs a deposit, and then two withdrawals, printing the balance after each operation.

```
1 class BankAccount {
2     private int accountNumber;
3     private String holderName;
4     private double balance;
5     BankAccount(int accountNumber, String holderName, double balance) {
6         this.accountNumber = accountNumber;
7         this.holderName = holderName;
8         this.balance = balance;
9     }
10    void deposit(double amount) {
11        if(amount > 0) {
12            balance += amount;
13        } else {
14            System.out.println("Invalid Deposit");
15        }
16    }
17    void withdraw(double amount) {
18        if(amount > 0 && amount <= balance) {
19            balance -= amount;
20        } else {
21            System.out.println("Invalid Withdrawal");
22        }
23    }
24    double getBalance() {
25        return balance;
26    }
27 }
28 class Main {
29     public static void main(String[] args) {
30         BankAccount b = new BankAccount(101, "Ram", 10000);
31         b.deposit(2000);
32         b.withdraw(3000);
33         System.out.println("Balance: " + b.getBalance());
34         b.withdraw(20000);
35     }
36 }
```

The output shows the initial balance of 9000.0, followed by an invalid withdrawal message, and then the final balance of 10000.0 after a successful deposit and withdrawal. The code execution is successful.

5. University Admission System



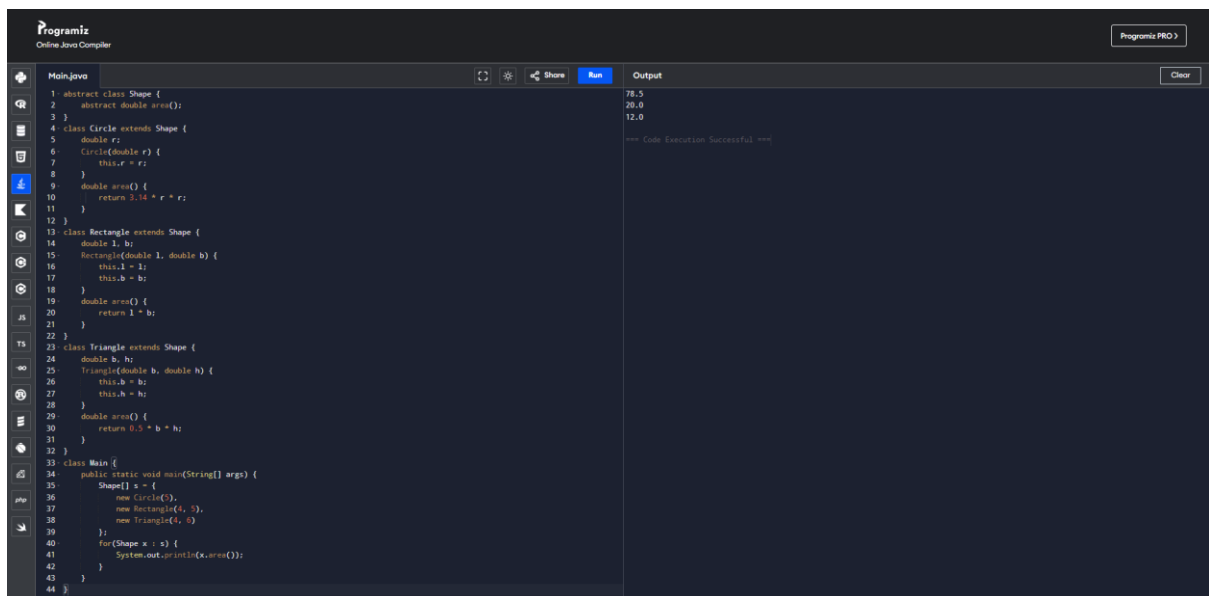
```
1- class Admission {
2-     double calculateFee(int fee) {
3-         return fee;
4-     }
5-     double calculateFee(int fee, int extra) {
6-         return fee + extra;
7-     }
8-     double calculateFee(int fee, double scholarship) {
9-         return fee - scholarship;
10-    }
11- }
12- class Main {
13-     public static void main(String[] args) {
14-         Admission a = new Admission();
15-         System.out.println("UG: " + a.calculateFee(50000));
16-         System.out.println("PG: " + a.calculateFee(50000, 20000));
17-         System.out.println("Scholarship: " + a.calculateFee(50000, 10000.0));
18-     }
19- }
```

Output:

```
UG: 50000.0
PG: 70000.0
Scholarship: 40000.0

=== Code Execution Successful ===
```

6. Shape Area Calculator



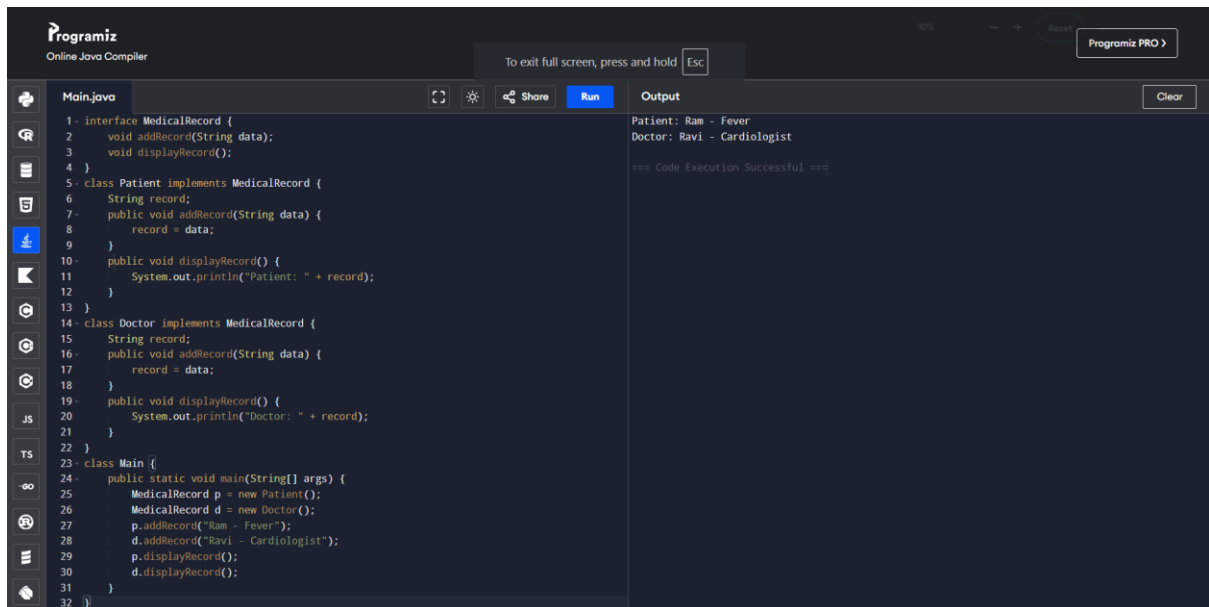
```
1- abstract class Shape {
2-     abstract double area();
3- }
4- class Circle extends Shape {
5-     double r;
6-     Circle(double r) {
7-         this.r = r;
8-     }
9-     double area() {
10-        return 3.14 * r * r;
11-    }
12- }
13- class Rectangle extends Shape {
14-     double l, b;
15-     Rectangle(double l, double b) {
16-         this.l = l;
17-         this.b = b;
18-     }
19-     double area() {
20-         return l * b;
21-     }
22- }
23- class Triangle extends Shape {
24-     double b, h;
25-     Triangle(double b, double h) {
26-         this.b = b;
27-         this.h = h;
28-     }
29-     double area() {
30-         return 0.5 * b * h;
31-     }
32- }
33- class Main {
34-     public static void main(String[] args) {
35-         Shape[] s = {
36-             new Circle(5),
37-             new Rectangle(4, 5),
38-             new Triangle(4, 6)
39-         };
40-         for(Shape x : s) {
41-             System.out.println(x.area());
42-         }
43-     }
44- }
```

Output:

```
78.5
20.0
12.0

=== Code Execution Successful ===
```

7. Hospital Management System



The screenshot shows the Programiz Online Java Compiler interface. The code defines an interface `MedicalRecord` with methods `addRecord` and `displayRecord`. Two classes, `Patient` and `Doctor`, implement this interface. The `Main` class creates instances of `Patient` and `Doctor`, adds records for them, and displays the records. The output shows the records for Patient (Ram - Fever) and Doctor (Ravi - Cardiologist).

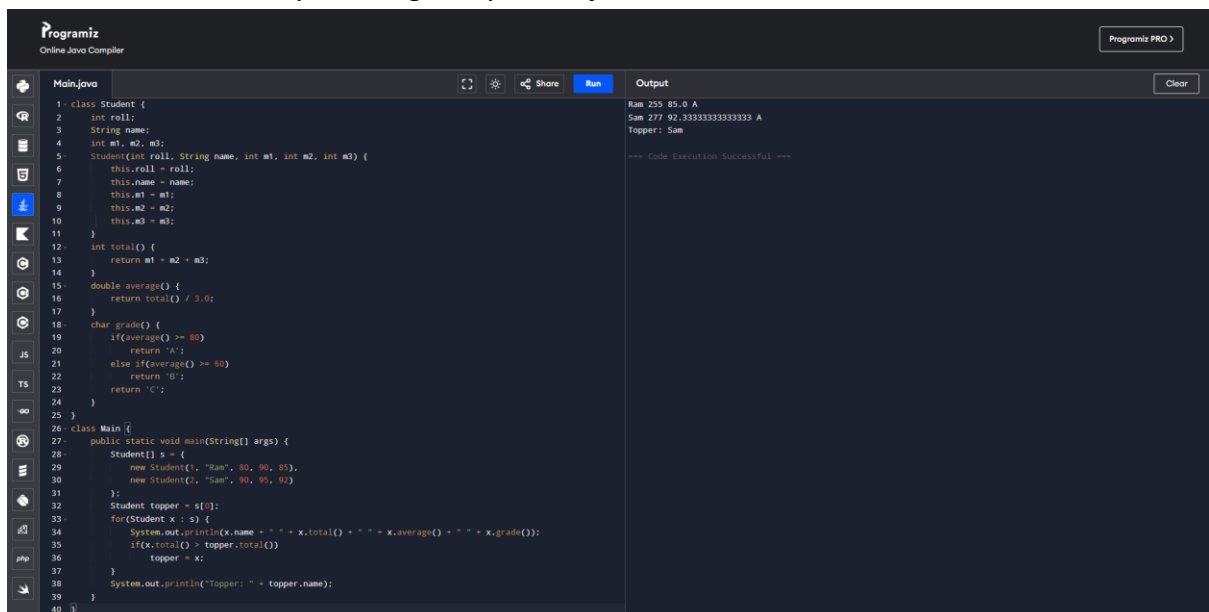
```
1- interface MedicalRecord {
2-     void addRecord(String data);
3-     void displayRecord();
4- }
5- class Patient implements MedicalRecord {
6-     String record;
7-     public void addRecord(String data) {
8-         record = data;
9-     }
10-    public void displayRecord() {
11-        System.out.println("Patient: " + record);
12-    }
13- }
14- class Doctor implements MedicalRecord {
15-     String record;
16-     public void addRecord(String data) {
17-         record = data;
18-     }
19-    public void displayRecord() {
20-        System.out.println("Doctor: " + record);
21-    }
22- }
23- class Main {
24-    public static void main(String[] args) {
25-        MedicalRecord p = new Patient();
26-        MedicalRecord d = new Doctor();
27-        p.addRecord("Ram - Fever");
28-        d.addRecord("Ravi - Cardiologist");
29-        p.displayRecord();
30-        d.displayRecord();
31-    }
32- }
```

Output:

```
Patient: Ram - Fever
Doctor: Ravi - Cardiologist

=== Code Execution Successful ===
```

9. Student Result Analyzer using Arrays of Objects



The screenshot shows the Programiz Online Java Compiler interface. The code defines a `Student` class with attributes `roll`, `name`, and `marks` (an array of three marks). It includes methods for calculating the total, average, and grade. The `Main` class creates an array of `Student` objects, calculates the total and average for each, and determines the topper (the student with the highest total marks).

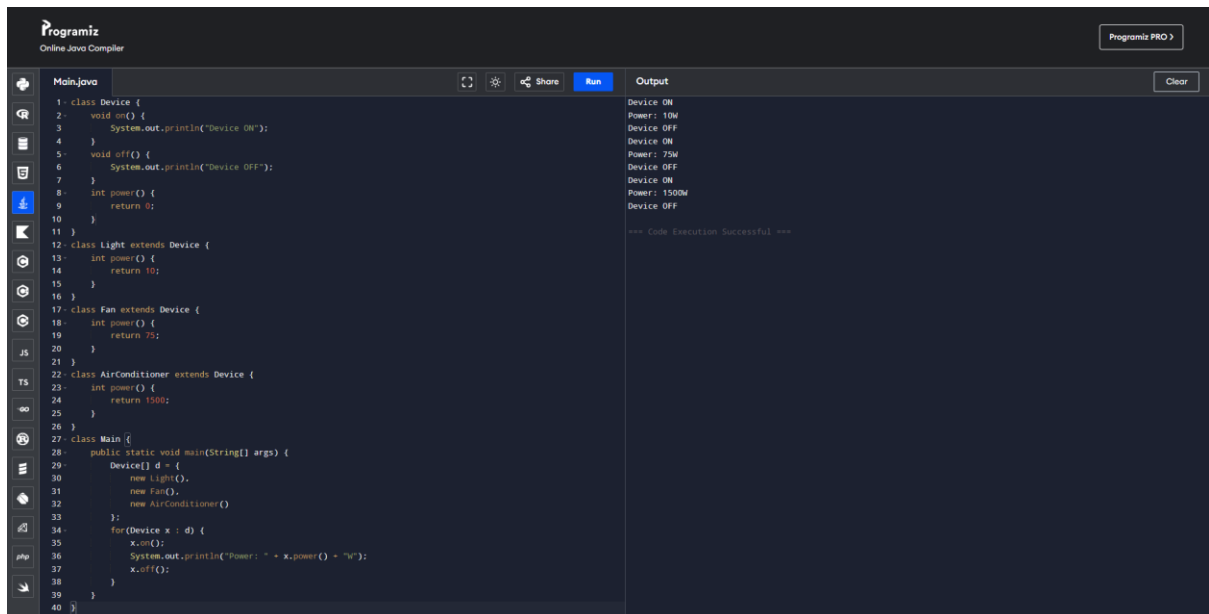
```
1- class Student {
2-     int roll;
3-     String name;
4-     int m1, m2, m3;
5-     Student(int roll, String name, int m1, int m2, int m3) {
6-         this.roll = roll;
7-         this.name = name;
8-         this.m1 = m1;
9-         this.m2 = m2;
10-        this.m3 = m3;
11-    }
12-    int total() {
13-        return m1 + m2 + m3;
14-    }
15-    double average() {
16-        return total() / 3.0;
17-    }
18-    char grade() {
19-        if(average() >= 80)
20-            return 'A';
21-        else if(average() >= 60)
22-            return 'B';
23-        return 'C';
24-    }
25- }
26- class Main {
27-    public static void main(String[] args) {
28-        Student[] s = {
29-            new Student(1, "Ram", 80, 90, 85),
30-            new Student(2, "Sam", 90, 95, 92)
31-        };
32-        Student topper = s[0];
33-        for(Student x : s) {
34-            System.out.println(x.name + " " + x.total() + " " + x.average() + " " + x.grade());
35-            if(x.total() > topper.total())
36-                topper = x;
37-        }
38-        System.out.println("Topper: " + topper.name);
39-    }
40- }
```

Output:

```
Ram 255 85.0 A
Sam 277 92.33333333333333 A
Topper: Sam

=== Code Execution Successful ===
```

10. Smart Home Device Controller using Hierarchical Inheritance



The screenshot displays the Programiz Online Java Compiler interface. The editor shows a Java program with hierarchical inheritance. The `Device` class has `on()`, `off()`, and `power()` methods. `Light`, `Fan`, and `AirConditioner` classes inherit from `Device` and override the `power()` method. The `Main` class creates instances of these devices and calls `on()`, `off()`, and `power()` methods. The output shows the execution results, including the power status and power consumption for each device.

```
1 class Device {
2     void on() {
3         System.out.println("Device ON");
4     }
5     void off() {
6         System.out.println("Device OFF");
7     }
8     int power() {
9         return 0;
10    }
11 }
12 class Light extends Device {
13     int power() {
14         return 10;
15     }
16 }
17 class Fan extends Device {
18     int power() {
19         return 75;
20     }
21 }
22 class AirConditioner extends Device {
23     int power() {
24         return 1500;
25     }
26 }
27 class Main {
28     public static void main(String[] args) {
29         Device[] d = {
30             new Light(),
31             new Fan(),
32             new AirConditioner()
33         };
34         for(Device x : d) {
35             x.on();
36             System.out.println("Power: " + x.power() + "W");
37             x.off();
38         }
39     }
40 }
```

Output:

```
Device ON
Power: 10W
Device OFF
Device ON
Power: 75W
Device OFF
Device ON
Power: 1500W
Device OFF

=== Code Execution Successful ===
```